PocketSmart AI

AI-Powered Budget Planning & Personalized Recommendations

Project Documentation

FastAPI • Google Gemini • Jinja2 • JWT • Python

1. Project Description

PocketSmart AI is a web-based application that uses artificial intelligence to generate personalized budget plans and product recommendations across three life domains: home interiors, party planning, and jewelry selection. Built with FastAPI and integrated with Google's Gemini model, the application simplifies planning and shopping within a defined budget.

The application accepts user inputs such as budget, goal, occasion, and preferences and generates customized, category-wise recommendations with shopping links to relevant Indian platforms. For the jewelry planner, users can optionally upload an outfit image so the AI can analyze colors, style, and formality and suggest coordinated jewelry.

PocketSmart AI includes JWT-based authentication, session tracking, recommendation history, a responsive HTML interface, and interactive API documentation through FastAPI's Swagger UI.

2. Scenarios

Scenario 1 — Home Interior Planning

A user enters a total budget and requirements such as the number of lights, fans, furniture pieces, dining tables, and rooms. The AI generates a category-wise budget breakdown with product suggestions, Indian brands, estimated prices, search terms, and shopping links.

Scenario 2 — Party Planning

A user enters a budget, party type, guest count, and required services such as catering, decoration, and entertainment. The system allocates the budget across relevant categories and provides service or vendor suggestions.

Scenario 3 — Jewelry Selection

A user enters a budget, occasion, and style preferences and may upload an outfit image. Gemini performs multimodal analysis of the outfit and suggests jewelry that coordinates with its colors, style, and formality.

Scenario 4 — Recommendation History

Users can access previously generated plans through a dedicated history dashboard. Each record preserves the original input, AI-generated recommendation, timestamp, recommendation type, and shopping links.

3. Technical Architecture

PocketSmart AI follows a modular, layered architecture consisting of the following components:

- User / Browser — Provides the interactive web interface.
- FastAPI Backend — Handles routing, validation, authentication, session control, and template rendering.
- Jinja2 Templates — Render the landing page, authentication pages, planner forms, dashboard, results, and history.
- Planner Modules — Separate logic for Home, Party, and Jewelry planning.
- AI Layer — Sends structured prompts and multimodal inputs to the Google Gemini model.
- Recommendation Service Layer — Orchestrates prompt generation, AI response parsing, fallback handling, and shopping-link enrichment.
- Authentication & Session Layer — Uses JWT authentication and in-memory data structures for users, sessions, and recommendation history.
- Image Storage — Stores uploaded outfit images in the static uploads directory for jewelry analysis.

Frontend Responsibilities

- Provide landing, registration, login, dashboard, planner, and history pages.
- Collect user inputs through responsive HTML forms.
- Submit planner requests using JavaScript.
- Render AI-generated recommendations in structured cards and tables.
- Provide access to previous recommendation details.

Backend Responsibilities

- Receive and validate user input using Pydantic schemas.
- Authenticate users using JWT tokens stored in HTTP-only cookies.
- Call the recommendation service layer.
- Store generated recommendations in user history.
- Render dynamic Jinja2 templates.
- Handle uploaded outfit images for multimodal jewelry recommendations.

AI Integration Points

- Home planner → AI-generated budget breakdown and product recommendations.
- Party planner → AI-generated event budget allocation and service recommendations.
- Jewelry planner → Multimodal outfit analysis and coordinated jewelry recommendations.

4. Prerequisites

1. FastAPI framework knowledge
2. Gemini API familiarity
3. HTML, CSS, JavaScript, and Jinja2 template skills
4. Python programming proficiency
5. Git and version-control basics
6. JWT and session-management basics
7. Python environment management using pip and virtualenv
8. Uvicorn ASGI server knowledge

5. Milestones Overview

9. Model Selection and Architecture — Select the AI model, define the architecture, and configure the development environment.
10. Core Functionalities Development — Implement the planners, FastAPI backend, AI integration, authentication, and history storage.
11. Route Development — Implement and connect application routes with the service and data layers.
12. Frontend Development — Design responsive pages and integrate dynamic Jinja2 rendering.
13. Deployment and Testing — Configure local deployment, verify application behavior, and prepare for future cloud deployment.

6. Milestone 1 — Model Selection and Architecture

Activity 1.1 — Research and Select the Appropriate Generative AI Model

Project Requirements

- Generate structured, category-wise budget breakdowns.
- Provide concise and actionable shopping recommendations.
- Support multimodal inputs for the jewelry planner.
- Respond quickly enough for web-based interaction.
- Return consistent, structured output that can be processed by the backend.

Alternative Approaches Evaluated

- Rule-based templates for static recommendations.
- Hugging Face text-generation models such as T5 and BART.
- DistilBERT for potential classification tasks.

These approaches were less suitable for the complete project because they required additional hosting, fine-tuning or prompt engineering, and did not provide the required multimodal workflow for outfit analysis.

Selected Model

The project selected Google's Gemini model because it supports structured text generation, multimodal input processing, API-based integration, and fast responses suitable for an interactive web application.

Activity 1.2 — Define the Application Architecture

- Frontend: HTML, CSS, Vanilla JavaScript, and Jinja2.
- Backend: FastAPI.
- AI Layer: Google Gemini API.
- Service Layer: Recommendation orchestration and response processing.
- Authentication: JWT with password hashing.
- Session and History Store: In-memory Python dictionaries.
- Image Handling: Pillow and static uploads.

Activity 1.3 — Development Environment Setup

- Install Python 3.11 or higher and pip.

- Create and activate a project virtual environment.
- Install the required project dependencies.
- Pin bcrypt to a compatible version when required by the authentication stack.
- Create a .env file for API keys and application secrets.
- Run the application with Uvicorn in development mode.
- Use the local application URL and FastAPI Swagger documentation for testing.

7. Project Directory Structure

The project is organized into a top-level application folder containing configuration files and an app package. The following structure separates validation, routing, services, utilities, templates, static assets, and tests.

```
PocketSmart-AI/
├── requirements.txt
├── .env
├── .gitignore
├── README.md
├── app/
│   ├── models/
│   ├── routes/
│   ├── services/
│   └── utils/
├── templates/
│   ├── base.html
│   ├── landing.html
│   ├── login.html
│   ├── register.html
│   ├── dashboard.html
│   ├── home_planner.html
│   ├── party_planner.html
│   ├── jewelry_planner.html
│   └── history.html
├── static/
│   ├── css/
│   ├── js/
│   └── uploads/
└── tests/
```

Package Responsibilities

- models/ — Pydantic request schemas and in-memory user data structures.
- routes/ — Authentication, planner, and history route modules.
- services/ — Gemini API utilities and recommendation service logic.
- utils/ — Authentication helpers and general-purpose utility functions.
- templates/ — Jinja2 HTML templates.
- static/ — CSS, JavaScript, and uploaded outfit images.
- tests/ — Automated tests using pytest.

8. Milestone 2 — Core Functionalities Development

Activity 2.1 — Develop the Core Functionalities

Home Budget Plan Generation

The home planner accepts the total budget, number of lights, fans, furniture pieces, dining tables, rooms, and additional requirements. The AI generates a structured category-wise plan containing budget allocations, individual items, descriptions, estimated prices, quantities, search terms, shopping links, and cost-saving suggestions.

Party Budget Plan Generation

The party planner uses event type, guest count, budget, and required services to generate a practical allocation across venue, catering, decoration, entertainment, and contingency. Recommendations can include service or vendor search terms and relevant shopping or booking links.

Multimodal Jewelry Recommendations

- Outfit analysis — identifies visible colors, style, and formality.
- Jewelry recommendations — suggests suitable jewelry types, descriptions, styles, and estimated prices.
- Styling tips — provides practical coordination suggestions.

Session and History Storage

Each generated recommendation is stored against the authenticated user and contains:

- Unique recommendation ID
- Timestamp
- Recommendation type
- Original user input
- Full AI-generated result
- Human-readable summary

The history list is limited to the most recent 50 entries per user.

Recommendation History Dashboard

The history page displays recommendation cards using Jinja2 loops. Users can open a recommendation to review its original input and complete generated output.

Activity 2.2 — FastAPI Backend and User Input Processing

Input Schemas

- HomeBudgetInput — Validates home-planning inputs.
- PartyBudgetInput — Validates party-planning inputs.
- JewelryBudgetInput — Validates jewelry-planning inputs and associated form data.

Authentication

Authentication is enforced using JWT tokens stored in HTTP-only cookies. Passwords are stored using secure password hashing rather than plain text.

AI Integration

- Home planner route → Home recommendation service → Gemini.

- Party planner route → Party recommendation service → Gemini.
- Jewelry planner route → Multimodal jewelry service → Gemini.

Safety and Fallback Handling

- Enrich missing shopping links where possible.
- Validate and normalize AI-generated data.
- Calculate and display cost totals.
- Return fallback responses when AI output is incomplete.
- Maintain consistent response structures across planners.

9. Milestone 3 — Route Development

Activity 3.1 — Main Application Logic

Core Routes

Method	Endpoint	Purpose
GET	/	Landing page
GET	/register	Registration page
POST	/register	Create a new account
GET	/login	Login page
POST	/token	Authenticate and issue JWT
GET	/logout	End the current session
GET	/dashboard	Authenticated user dashboard
GET	/home_planner	Home planner form
POST	/home-budget	Generate home recommendations
GET	/party_planner	Party planner form
POST	/party-budget	Generate party recommendations
GET	/jewelry_planner	Jewelry planner form
POST	/jewelry-budget	Generate jewelry recommendations
GET	/history	Recommendation history page

Route Processing Flow

14. Authenticate the current user.
15. Receive form or JSON input.
16. Validate input against the appropriate schema.
17. Pass validated data to the recommendation service.
18. Call the Gemini-powered AI function.
19. Validate or normalize the AI response.
20. Save the generated recommendation to user history.
21. Return the result to the corresponding frontend page.

Jewelry Image Processing

- Accept an optional multipart image upload.
- Validate the uploaded file type.
- Store the image in the static uploads directory.
- Pass the image and text requirements to the multimodal AI service.
- Display the outfit analysis and jewelry recommendations.

10. Milestone 4 — Frontend Development

Activity 4.1 — User Interface Design

- Responsive navigation bar with PocketSmart branding.
- Landing page with hero section and feature cards.
- Authentication pages for registration and login.
- Post-login dashboard with planner cards and recent activity.
- Dedicated Home, Party, and Jewelry planner forms.
- History dashboard with recommendation cards and detailed views.
- Responsive CSS using Flexbox, CSS Grid, media queries, shadows, rounded corners, and gradients.

Activity 4.2 — Dynamic Jinja2 Templates

- Use a shared base template for consistent layout.
- Pass authenticated-user data into protected templates.
- Bind planner forms to backend field names.
- Render recommendation results dynamically.
- Loop through recommendation history and display each item as a card.
- Provide detailed recommendation views without requiring the user to regenerate the plan.

11. Milestone 5 — Deployment and Testing

Activity 5.1 — Local Deployment

- Create and activate the Python virtual environment.
- Install dependencies from requirements.txt.
- Configure environment variables in .env.
- Start the FastAPI application using Uvicorn.
- Verify that templates, static files, authentication, and AI services are available.

Activity 5.2 — Testing and Verification

The following application areas should be verified during local testing:

- Home page and navigation.
- User registration and login.
- Dashboard and authenticated access.
- Home budget generation.
- Party budget generation.
- Jewelry recommendation generation with and without an outfit image.
- Recommendation history and detailed views.
- Health-check endpoint.
- FastAPI Swagger documentation.

12. Application Pages

Home Page

The homepage presents the PocketSmart AI brand, application value proposition, feature cards for the three planners, testimonials, and calls to action.

Login and Registration

The authentication interface allows users to create accounts and sign in before accessing personalized planning features.

Dashboard

The dashboard greets the authenticated user and provides access to the Home, Party, and Jewelry planners. It also surfaces recent recommendation activity and the full history.

Home Budget Planner

Inputs include total budget, number of lights, fans, furniture pieces, dining tables, selected rooms, and additional requirements.

Party Budget Planner

Inputs include budget, party type, guest count, and required services such as catering, decoration, and entertainment.

Jewelry Budget Planner

Inputs include budget, occasion, style preferences, and an optional outfit image. The image upload supports multimodal recommendation generation.

Recommendation Results

Result pages display budget summaries, category breakdowns, recommended items, estimated prices, descriptions, and relevant shopping or booking links.

History Page

The history dashboard presents previous recommendations as cards. Each record can be opened to inspect the original input and complete generated recommendation.

13. Technology Stack

Layer	Technology
Backend Framework	FastAPI
ASGI Server	Uvicorn
AI Model	Google Gemini
AI SDK	Google Generative AI SDK
Templating	Jinja2
Authentication	JWT (python-jose) with bcrypt password hashing
Validation	Pydantic
Image Handling	Pillow
Session / History Storage	In-memory Python dictionaries
Frontend	HTML5, CSS3, Vanilla JavaScript
Testing	pytest
Environment Configuration	python-dotenv
Deployment	Local Uvicorn; suitable for future cloud deployment
Python Version	3.11 or higher

14. API Endpoints Summary

Method	Endpoint	Purpose
GET	/	Landing page
GET	/login	Login page
GET	/register	Registration page
POST	/register	Create new account
POST	/token	Get JWT token
GET	/logout	End session
GET	/dashboard	User dashboard
GET	/home_planner	Home planner form
POST	/home-budget	Generate home recommendations
GET	/party_planner	Party planner form
POST	/party-budget	Generate party recommendations
GET	/jewelry_planner	Jewelry planner form
POST	/jewelry-budget	Generate jewelry recommendations
GET	/history	Past recommendations
GET	/history-data	Recommendation list as JSON
GET	/recommendation-details/{id}	Full details for one recommendation
GET	/health	Health check
GET	/docs	Interactive API documentation

15. Scenarios vs. Endpoints

Scenario	User Action	Endpoint	AI Processing
Home Interior	Enter budget and room requirements	POST /home-budget	Gemini
Party Planning	Enter budget and event details	POST /party-budget	Gemini
Jewelry + Outfit	Enter budget, occasion, and optional image	POST /jewelry-budget	Gemini multimodal
History	View past recommendations	GET /history	Not required

16. Conclusion

PocketSmart AI demonstrates how generative AI can be integrated into a practical web application to support budget planning and personalized recommendations. The project combines FastAPI, Jinja2, JWT authentication, AI-powered recommendation services, multimodal image analysis, and recommendation history in a single platform.

The three planners address different everyday needs: home interior planning, party planning, and jewelry selection. By accepting user-defined budgets and preferences, the application produces structured recommendations that can be presented through an interactive web interface.

The modular architecture also provides a foundation for future improvements such as persistent database storage, live product-price retrieval, stronger recommendation validation, cloud deployment, and expanded planner categories.